

Chapter 2: Types, Operators, and Expressions

Relational graph edition. Pull-shaped. Top-down.

How to use this document

Red boxes = checkpoints (answer in chat). Green boxes = practice with inline answers. Blue box = target problem. Walk the edges in order.

Big picture

Every value in C has a type. Types determine how many bytes are used, what operations are legal, and what happens when you mix types in an expression. Operators transform values. This chapter is the type system and operator reference -- the rules of the road.

Target problem

Write a function `unsigned int bitcount(unsigned int x)` **that counts the number of 1-bits in x using bitwise operators. Then write a test that demonstrates type conversion by printing the count as int, float, and char.**

The Graph

Node inventory

Node	Size (typical)	Range / Purpose
char	1 byte	0-255 unsigned, or -128 to 127 signed
int	4 bytes	-2B to +2B (32 bits)
long	8 bytes	At least 32 bits, often 64
float	4 bytes	~7 decimal digits precision
double	8 bytes	~15 decimal digits precision
unsigned	modifier	No sign bit -- doubles positive range
enum	named ints	enum bool { NO, YES }; -- NO=0, YES=1
const	modifier	Value cannot be changed after init

Operator edge table

Category	Operators	Key rule
Arithmetic	+ - * / %	Integer / truncates. % is remainder (ints only)
Relational	> < >= <= == !=	Result is int: 1 (true) or 0 (false)
Logical	&& !	Short-circuit: stops evaluating when result is known
Bitwise	& ^ ~ << >>	Operate on individual bits. & masks, sets, ^ toggles, ~ inverts
Assignment	= += -= *= /= %= &= = ^= <<= >>=	x op= y means x = x op y
Increment	++x x++ --x x--	Prefix: increment then use. Postfix: use then increment
Ternary	a ? b : c	If a is nonzero, result is b; else c

Implicit conversion chain (memorize this)

char → int → long → float → double

When two operands differ in type, the narrower one is promoted to the wider one. This happens silently. Assigning a wider type to a narrower type truncates (may lose data).

Zoom in -- one edge at a time

Edge 1: Data types and sizes

Use `sizeof(type)` to check actual size on your machine. The only guarantees: `sizeof(char) == 1`, `sizeof(short) <= sizeof(int) <= sizeof(long)`. Header `<limits.h>` defines exact ranges (`INT_MAX`, `CHAR_BIT`, etc).

```
printf("int: %zu bytes, max: %d\n", sizeof(int), INT_MAX);
```

Problem 2.1: Write a program that prints the size of `char`, `int`, `long`, `float`, and `double` on your machine.

Answer: Five `printf` calls using `sizeof(type)` with `%zu` format.

Edge 2: Constants -- all the forms

Form	Example	Type
Decimal	42	int
Octal	052	int (leading 0)
Hex	0x2A	int (leading 0x)
Character	'*'	int (ASCII value 42)
String	"hello"	char array + \0
Float	3.14	double (not float!)
Float (explicit)	3.14f	float
Long	42L	long
Unsigned	42U	unsigned int
Enum	enum { RED, GREEN=5, BLUE }	RED=0, GREEN=5, BLUE=6

Problem 2.2: What type is each: 077, 0xFF, 'A', 3.14, 3.14f, 100L?

Answer: `int(octal 63)`, `int(hex 255)`, `int(65)`, `double`, `float`, `long`.

Edge 3: Arithmetic operators

`%` (modulus) works only on integers. `/` truncates when both operands are integers: `7/2 == 3`, not 3.5. To get float division, make at least one operand float: `7.0/2 == 3.5`.

Problem 2.3: What is the value of: `13/5`, `13%5`, `13.0/5`?

Answer: 2, 3, 2.6

Edge 4: Relational, logical, short-circuit

&& and || short-circuit: in a && b, if a is 0 then b is never evaluated. In a || b, if a is nonzero then b is never evaluated. This is a guarantee, not an optimization. You can rely on it.

Problem 2.4: In `if (p != NULL && *p == 'x')`, why is the order of the two tests critical?

Answer: Short-circuit. If p is NULL, the second test (*p) is never evaluated, avoiding a null pointer dereference.

CHECKPOINT A

Without looking above: what is the implicit promotion chain? What does `7/2` evaluate to, and why? What does short-circuit mean?

Edge 5: Type conversions

Implicit: narrower type auto-promotes to wider in mixed expressions. Explicit: (type)expr forces conversion. Assigning double to int truncates toward zero. Assigning negative to unsigned wraps around.

```
int n = 3;
double avg = (double)n / 2; /* cast n to double first, then divide */
int trunc = (int)3.99; /* trunc == 3, fractional part dropped */
```

Problem 2.5: What does unsigned u = -1; produce?

Answer: UINT_MAX (all bits set to 1). Negative values wrap around in unsigned.

Edge 6: Increment/decrement -- prefix vs postfix

++i: increment i, then use the new value. i++: use the current value of i, then increment. In standalone statements (i++;) there is no difference. In expressions, it matters.

```
int i = 5;
int a = ++i; /* i becomes 6, a gets 6 */
int b = i++; /* b gets 6 (current i), then i becomes 7 */
```

Problem 2.6: If int x = 3;, what is the value of x++ + ++x?

Answer: Undefined behavior. Never modify the same variable twice in one expression without a sequence point.

Edge 7: Bitwise operators

Operator	Name	What it does
&	AND	Both bits must be 1 to produce 1
	OR	Either bit being 1 produces 1
^	XOR	Exactly one bit being 1 produces 1
~	NOT	Flips every bit
<<	Left shift	Shifts bits left, fills with 0s (multiplies by 2)
>>	Right shift	Shifts bits right (divides by 2, sign-extension varies)

Common pattern -- masking: x & 0xFF extracts the lowest 8 bits. x | 0x80 sets bit 7. x & ~0x80 clears bit 7.

Problem 2.7: Write an expression that extracts bits 4-7 (4 bits) from an unsigned int x.

Answer: (x >> 4) & 0xF -- shift right 4 to align, then mask.

Edge 8: Precedence (the trap)

When in doubt, use parentheses. The most common trap: `&` and `|` have LOWER precedence than `==`. So `x & mask == 0` is parsed as `x & (mask == 0)`, which is almost certainly wrong. Write `(x & mask) == 0` instead.

Problem 2.8: What does `2 << 3 + 1` evaluate to?

Answer: `2 << (3+1) = 2 << 4 = 32`. `+` has higher precedence than `<<`.

CHECKPOINT B

Write a function `unsigned rightrot(unsigned x, int n)` that returns `x` rotated right by `n` bit positions. Bits that fall off the right end reappear on the left.

Solving the target problem

Walk: E7 (bitwise) + E5 (type conversion) + E6 (increment)

```
#include <stdio.h>

unsigned int bitcount(unsigned int x) {
    unsigned int count = 0;
    for (; x != 0; x >>= 1) /* E7: shift right */
        count += x & 1; /* E7: mask lowest bit */
    return count;
}

int main(void) {
    unsigned int val = 0xAB; /* binary: 10101011, expect 5 */
    unsigned int n = bitcount(val);
    printf("int: %u\n", n); /* E5: no conversion */
    printf("float: %.1f\n", (double)n); /* E5: explicit cast */
    printf("char: '%c'\n", (char)('0'+n)); /* E5: int to char */
    return 0;
}
```

CHECKPOINT C

Rewrite bitcount using the trick: $x \&= (x-1)$ clears the rightmost 1-bit. Why is this faster?

Final exam

Problem 1: Write a function `int htoi(char s[])` that converts a hex string to int.

Problem 2: Write a function `unsigned invert(unsigned x, int p, int n)` that inverts `n` bits starting at position `p`.

Problem 3: Write a function `unsigned setbits(unsigned x, int p, int n, unsigned y)` that sets `n` bits of `x` starting at `p` to the rightmost `n` bits of `y`.

Problem 4: Write an expression equivalent to `x *= (1 << n)` using only shift operators.

Problem 5: Explain why `strlen(s) - strlen(t)` can produce a wrong result when `s` is shorter than `t`, and how to fix it.

CHECKPOINT D

Solve at least 3 of the 5 problems. Report in chat.

Answer key

1. htoi: Walk E2 (hex constants) + E5 (char to int). Loop through chars, multiply accumulator by 16, add digit value. 'a'-'f' = 10-15.
2. invert: Walk E7. Create mask of n bits at position p with shifts, XOR with x.
3. setbits: Walk E7. Clear target bits with AND+NOT mask, OR in the new bits shifted to position.
4. $x \ll n$. Left shift by n is multiplication by 2^n .
5. strlen returns size_t (unsigned). If s is shorter, the subtraction wraps to a huge positive number. Fix: cast to int, or compare lengths before subtracting.